

Discovery Executive Summary

Project: discovery-pingCRM-7Aug · Generated: 07/08/2026, 17:05:22

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating
1	Architecture & Design Analysis	—

1. Architecture & Design Analysis

EXECUTIVE SUMMARY

This is a Laravel 11 + Inertia/React application (PingCRM base) that has been overgrown with a large, deliberately-legacy "IVR" subsystem. Architectural health is **High Risk**: 83 single-action IVR controllers each carry ~759 lines of inline business logic, raw string-concatenated `DB::select` queries, `extract($request->all())`, hard-coded tenant IDs, and `new SomethingGodService()` instantiation — so controllers, persistence, and domain rules are fused into one untestable layer. A Repository tier (12 classes) and a Service tier (12 `*GodService` classes) both exist but are architecturally hollow: the repositories are referenced by zero call-sites (dead abstraction) and the services are god-classes with 45 unrelated public methods, mutable `static` state, and embedded secrets. There is no Dependency Injection wiring at all (`AppServiceProvider` binds nothing; `Model::unguard()` is global), so nothing can be substituted or mocked. On the frontend, 916 components repeat the same pathology in React terms — 374 components issue inline `fetch()` calls to hard-coded URLs, 147 legacy class components mix paradigms, and 8 near-identical 1,101-line "formatter" modules duplicate the same logic. The dominant risks are **change amplification** (one schema or rule change touches dozens of controllers, services, repos and components at once) and **hidden coupling** (reporting reads other domains' operational tables directly), both of which will make any modernization or team scaling extremely costly until bounded contexts, a real service layer, and DI are introduced.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	<span class="rating	<span class="rating rating-	<span class="rating	Measured
---	---------	-------------	---------------------	-----------------------------	---------------------	----------

			rating-good">Good	moderate">Moderate	rating-high-risk">High Risk	
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	~ 683 LOC avg (81 of 91 > 300 LOC; IVR = 759 LOC each)
H2	Missing Service Layer	Controllers accessing repos/models directly	<10	10–20	>20	83 controllers with inline DB/model access + rules
H3	Missing Repository Pattern	Direct DB access points outside repositories	<10	10–20	>20	95 files use <code>DB::</code> directly; 12 repositories have 0 references
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0 cycles observed
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	13 (5 backend helpers + 8 frontend dup formatters)
H6	Direct SQL in Controllers	ORM/repository compliance %	>90%	60–90%	<60%	~ 9% compliant (83 of 91 controllers embed raw SQL)
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 backend > 1000 LOC, but 12 <code>*GodService</code> @ 45 methods + static state
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	4 (Reports, IvrlHub, LoadslvrModuleData, IvrlAccountContext)
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	~ 21% (3 of 14 tables shared, no ownership)
H10 (additional)	No Dependency Injection / Service Locator	Services resolved via <code>new</code>	0	1–10	>10	80 <code>new</code> <code>*GodService()</code> ; 0 container bindings
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	avg 139 LOC ; 134 of 522 pages (26%) > 300 LOC
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	374 components with inline <code>fetch()</code> /hard-coded URLs
F3	God / Oversized	Components >400 LOC	0	1–3	>3	1 (<code>Hub/Index.tsx</code>)

	Components					= 479 LOC); 134 more in 300–400 band
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	≤2 — components self-fetch into local state
F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	147 class components mixed with function components

No additional hotspots beyond H10 were observed.

1.4 Actions Required

Hotspot	Action	Rating	Priority
H1 Fat Controllers	Reduce the 83 IVR controllers to validate-and-delegate; extract logic into Application Services	High Risk	Critical
H2 Missing Service Layer	Introduce per-capability Application Services; move query/workflow assembly out of controllers	High Risk	Critical
H6 Direct SQL in Controllers	Move all <code>DB::</code> queries out of controllers into bound-parameter repositories; add CI grep gate	High Risk	Critical
H10 No Dependency Injection	Bind interfaces in <code>AppServiceProvider</code> , inject services, remove <code>new *GodService()</code> and <code>Model::unguard()</code>	High Risk	Critical
H3 Missing Repository Pattern	Give repositories interfaces + bound queries; route all persistence through them	High Risk	High
H5 Shared Utility Abuse	Replace 5 <code>LegacyIvr*</code> helpers with domain services; collapse 8 duplicate formatter modules into one	High Risk	High
H7 God Classes	Split each <code>*GodService</code> into domain service + repository + application service; remove static state/secret	High Risk	High
F2 Missing Frontend Service Layer	Introduce a typed API client + hooks; replace 374 inline <code>fetch()</code> /hard-coded URLs	High Risk	High
F5 Legacy Component Patterns	Codemod 147 class widgets to hooks; add error boundaries and a ban-class lint rule	High Risk	High
H8 Domain Boundary Violations	Introduce published read models / ACL; forbid cross-context table reads in Reports & Hub	Moderate	Medium
H9 Shared Database Coupling	Assign per-table ownership; serve reporting via an owned read model instead of live joins	Moderate	Medium

F1 Business Logic in Components	Move validation/transform into shared hooks/schema; split 300–400-LOC page templates	Moderate	Medium
F3 God / Oversized Components	Decompose <code>Hub/Index.tsx</code> (479 LOC) and the shared 392-LOC template into widgets + hooks	Moderate	Medium

1.5 Expected Outcomes

- **Testable, reusable business logic** — with Application/Domain Services and constructor DI in place, workflows can be unit-tested with mocked repositories and reused from HTTP, CLI and queued jobs instead of being copy-pasted across 83 controllers.
- **Schema and persistence freedom** — funnelling every `DB::` call through parameter-bound repositories removes the injection holes and lets the team rename/repartition `ivr_*` tables without editing dozens of hand-written query strings.
- **Independent, extractable domains** — bounded contexts with published interfaces and an anti-corruption layer let Queue, Call Routing, Analytics and Reporting evolve (and eventually extract) without silently breaking each other through shared tables.
- **A consistent, maintainable frontend** — a single typed API client plus hooks and function components collapses 374 inline fetches and 147 class widgets into one convention, so endpoint/auth changes are one-file edits and screens stop leaking timers and blanking on errors.
- **Sharply lower change amplification** — de-duplicating the 5 helpers and 8 formatter modules and enforcing CI gates (no `DB::` in controllers, no `React.Component`, duplication check) means a single rule change stops rippling through dozens of near-identical files. {"tftf_ms":7882,"tftf_stream_ms":7697,"time_to_request_ms":6140,"type":"result","duration_ms":491456,"uuid":"07a4177b-031e-4e68-a1fd-2230b3ef721b"}